

Relatório sobre a atividade “22 - Prática: Processamento de Linguagem Natural (NLP) (III)”

Lucas Gabriel Arenhardt

1. Introdução

Nesse módulo foi abordado o tema de Processamento de Linguagem Natural (NLP), com dois minicursos. O primeiro, intitulado “Natural Language Processing - (NLP Zero to Hero)”, aborda a manipulação de texto utilizando o TensorFlow. O segundo, com o nome de “Natural Language Processing with SpaCy & Python - Course for Beginners”, utiliza a biblioteca Spacy para realizar várias operações com textos. Os principais conhecimentos adquiridos estão resumidos abaixo.

2. Natural Language Processing (NLP Zero to Hero)

Nesse curso é ensinado a construir uma rede neural que seja capaz de prever a próxima palavra, dado um texto anterior.

Primeiramente é realizado o download do dataset, que consiste em um conjunto de textos irlandeses. Em seguida, é criado um objeto do tipo Tokenizer. O Tokenizer é uma ferramenta utilizada para dividir texto em unidades menores chamadas tokens. Esses tokens geralmente correspondem a palavras, números ou pontuações. Então, é utilizada a função “tokenizer.fit_on_texts(data)” para construir um dicionário, onde cada token possui um número inteiro representando-o.

Seguidamente, é montado um loop para transformar todas as palavras das sentenças do texto em uma sequência de inteiros, segundo o dicionário. Subsequências das sentenças são geradas para garantir que o treinamento seja bem sucedido. O padding (processo que consiste na colocação de zeros para que todos os tokens tenham o mesmo tamanho) é realizado no conjunto de sentenças. Também são criados os preditores e labels. Os labels são transformados para o formato one-hot vector, onde cada vetor tem uma dimensão igual ao número total de palavras. Isso significa que cada label é representado como um vetor binário com todos os elementos zero, exceto um, que indica a posição da palavra correspondente

A arquitetura da rede neural consiste em um modelo sequencial, onde a primeira camada é a camada de Embedding. O Embedding é responsável por transformar os tokens em vetores que capturam as relações semânticas entre os dados de entrada. Ou seja, elementos com significados ou características similares, terão vetores de embedding próximos. A próxima camada é uma camada recorrente bidirecional, isso permite que o modelo capture informações contextuais tanto do passado quanto do futuro em relação a um ponto específico na sequência, melhorando a compreensão de contextos complexos. A última camada é densa, com a função de ativação softmax e com o número de neurônios correspondente ao número total de palavras do texto inicial.

Layer (type)
embedding (Embedding)
bidirectional (Bidirectional)
dense (Dense)

Imagem 1 - Estrutura da Rede Neural.

Por fim, o novo texto é gerado por meio de um loop, onde cada palavra gerada pela rede neural é adicionada ao texto e este é passado como parâmetro novamente.

3. Natural Language Processing with SpaCy & Python - Course for Beginners

Esse curso é voltado para a utilização da biblioteca SpaCy, utilizada no processamento de linguagem natural.

3.1. O que é o SpaCy?

SpaCy é uma biblioteca de processamento de linguagem natural (NLP) de código aberto em Python, projetada para ser rápida e produtiva. Ela oferece ferramentas para realizar tarefas como tokenização, análise gramatical (parsing), reconhecimento de entidades nomeadas (NER), e vetorização de palavras (word embeddings). SpaCy é amplamente utilizada em aplicações de NLP devido à sua eficiência e facilidade de uso, além de ser compatível com modelos de aprendizado profundo e frameworks como TensorFlow e PyTorch.

3.2. Principais comandos e funções

O SpaCy conta com um amplo número de comandos e funções para a manipulação de texto. Abaixo estão dispostas e explicadas as suas principais:

- **spacy.load():** carrega um modelo de linguagem pré-treinado que pode incluir informações linguísticas como tokenização, análise gramatical, reconhecimento de entidades nomeadas e mais;
- **nlp(text):** é usado para processar e analisar texto. Quando o 'text' é passado para o objeto 'nlp', ele retorna um objeto 'Doc', que contém uma representação estruturada do texto original;

- **doc.text:** obtém-se a string original que foi processada pelo pipeline de spaCy. Esse é o texto exato que foi passado para o método `nlp()` e gerou o objeto Doc;
- **doc.ents:** retorna um objeto que contém todas as entidades reconhecidas no texto processado;
- **doc.sents:** retorna as sentenças do texto;
- **token.left_edge / token.right_edge:** retorna o token que está mais à esquerda/direita da subárvore do token analisado;
- **token.ent_type_:** retorna o rótulo da entidade associada ao token analisado;
- **token.ent_iob_:** indica se o token está no início, meio ou fim de uma entidade maior;
- **token.lemma_:** retorna a forma base da palavra;
- **token.morph:** retorna uma análise morfológica da palavra;
- **displacy.redner():** permite visualizar informações tais como dependências ou entidades no texto de forma gráfica;
- **spray.blank():** cria um objeto nlp sem nenhum pipeline;
- **nlp.add_pipeline():** adiciona um pipeline ao nlp;
- **nlp.analyze_pipes():** mostra os pipelines do nlp;
- **Matcher:** O Matcher é uma ferramenta para identificar padrões específicos em um texto, como sequências de palavras, frases, ou construções gramaticais, utilizando regras baseadas em tokens. Diferente de abordagens tradicionais de expressão regular que operam sobre strings, o Matcher do spaCy trabalha com objetos Doc e tokens, permitindo uma busca mais rica e contextualizada.

4. Conclusão

A atividade "22 - Prática: Processamento de Linguagem Natural (NLP) (III)" proporcionou uma imersão profunda em duas abordagens distintas para o processamento de linguagem natural: uma baseada no uso do TensorFlow e outra focada na biblioteca SpaCy. Com o primeiro curso, "Natural Language Processing - (NLP Zero to Hero)", foi possível desenvolver uma rede neural capaz de prever palavras em uma sequência de texto, abordando conceitos como tokenização, padding, embeddings e redes neurais recorrentes bidirecionais. No segundo curso, "Natural Language Processing with SpaCy & Python - Course for Beginners", o foco esteve na exploração da biblioteca SpaCy, que se mostrou uma ferramenta robusta e eficiente para tarefas de NLP, oferecendo comandos e funções versáteis para manipulação e análise de texto.

Esses conhecimentos são fundamentais para o desenvolvimento de soluções avançadas em NLP, seja na criação de modelos preditivos complexos ou na aplicação de ferramentas otimizadas para análise de grandes volumes de texto.